

Experiment - 09

Student Name: Om Tiwari

UID: 24BAI70372

Branch: CSE - AIML

Section/Group: 24AIT_KRG-1_A

Semester: 5

Date of Performance: 18/9/26

Subject Name: ADBMS

Subject Code: 24CSP-310

(A)

An organization stores employee details in the employee_info table. Create a PostgreSQL stored procedure that accepts an employee's gender as an input parameter and calculates the total number of employees belonging to that gender. The calculated count should be returned through an OUT parameter and should also be displayed as a message using RAISE NOTICE.

For example, if Male is passed to the procedure, it should count all employees whose gender is Male.

Column Name	Data Type	Description
id	SERIAL	Unique employee ID
name	VARCHAR(50)	Employee name
gender	VARCHAR(10)	Gender of employee
salary	NUMERIC(10,2)	Employee salary
city	VARCHAR(50)	Employee city

Solution:

```
CREATE TABLE employee_info (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    gender VARCHAR(10) NOT NULL,
    salary NUMERIC(10,2) NOT NULL,
    city VARCHAR(50) NOT NULL
);

INSERT INTO employee_info (name, gender, salary, city)
VALUES
('Alok', 'Male', 50000.00, 'Delhi'),
('Priya', 'Male', 60000.00, 'Mumbai'),
('Rajesh', 'Female', 45000.00, 'Bangalore');
```

```
( 'Sneha', 'Male', 55000.00, 'Chennai'),
( 'Anil', 'Male', 52000.00, 'Hyderabad'),
( 'Sunita', 'Female', 48000.00, 'Kolkata'),
( 'Vijay', 'Male', 47000.00, 'Pune'),
( 'Ritu', 'Male', 62000.00, 'Ahmedabad'),
( 'Amit', 'Female', 51000.00, 'Jaipur');
```

```
CREATE OR REPLACE PROCEDURE pr_count_gender(
    IN p_gender VARCHAR(10),
    OUT employee_count INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT COUNT(*)
    INTO employee_count
    FROM employee_info
    WHERE gender = p_gender;

    RAISE NOTICE 'Total employees with gender %: %',
        p_gender, employee_count;
END;
$$;

CALL pr_count_gender('Male', NULL);
```

(B)

An e-commerce company maintains product stock information in the products table and stores completed orders in the sales table. Create a PostgreSQL stored procedure that accepts a product name and required quantity as input parameters and processes the purchase only when sufficient stock is available.

The procedure should verify the available quantity of the requested product. If sufficient stock exists, it should retrieve the product code and price, calculate the total sale amount, insert a new order into the sales table, reduce the available stock, and increase the total quantity sold. If sufficient stock is not available, the procedure should not modify any data and should display an appropriate notification message.

The total sale amount should be calculated as:

$\text{Sale Price} = \text{Product Price} \times \text{Quantity Ordered}$

Tables:

Column Name	Data Type	Description
product_code	VARCHAR(10)	Unique product code
product_name	VARCHAR(100)	Product name
price	NUMERIC(10,2)	Price of one unit
quantity_remaining	INT	Current available stock
quantity_sold	INT	Total quantity sold

Column Name	Data Type	Description
order_id	SERIAL	Unique order ID
order_date	DATE	Date on which order was placed
product_code	VARCHAR(10)	Product purchased
quantity_order ed	INT	Quantity purchased
sale_price	NUMERIC(10,2)	Total order amount

Solution:

```

CREATE TABLE products (
    product_code VARCHAR(10) PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price NUMERIC(10,2) NOT NULL,
    quantity_remaining INT NOT NULL,
    quantity_sold INT DEFAULT 0
);

CREATE TABLE sales (
    order_id SERIAL PRIMARY KEY,
    order_date DATE NOT NULL,
    product_code VARCHAR(10) NOT NULL,
    quantity_ordered INT NOT NULL,
    sale_price NUMERIC(10,2) NOT NULL,
    FOREIGN KEY (product_code) REFERENCES products(product_code)
);

INSERT INTO products (product_code, product_name, price,
quantity_remaining, quantity_sold)
VALUES
('P001', 'iPhone 13 Pro Max', 109999.00, 10, 0),
('P002', 'Samsung Galaxy S23 Ultra', 99999.00, 8, 0),
('P003', 'iPad Air', 55999.00, 5, 0),
('P004', 'MacBook Pro 14"', 189999.00, 3, 0),
('P005', 'Sony WH-1000XM5 Headphones', 29999.00, 15, 0);

INSERT INTO sales (order_date, product_code, quantity_ordered,
sale_price)

```

```
VALUES
('2025-09-15', 'P001', 1, 109999.00),
('2025-09-16', 'P002', 2, 199998.00),
('2025-09-17', 'P003', 1, 55999.00),
('2025-09-18', 'P005', 2, 59998.00),
('2025-09-19', 'P001', 1, 109999.00);

CREATE OR REPLACE PROCEDURE pr_buy_products(
    IN p_product_name VARCHAR,
    IN p_quantity INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_product_code VARCHAR(20);
    v_price FLOAT;
    v_count INT;

BEGIN
    -- Step 1: Check if product exists and has enough quantity
    SELECT COUNT(*)
    INTO v_count
    FROM products
    WHERE product_name = p_product_name
    AND quantity_remaining >= p_quantity;

    -- Step 2: If sufficient stock
    IF v_count > 0 THEN

        -- Fetch product code and price
        SELECT product_code, price
        INTO v_product_code, v_price
        FROM products
        WHERE product_name = p_product_name;

        -- Insert a new record into the sales table
        INSERT INTO sales (order_date, product_code,
quantity_ordered, sale_price)
        VALUES (CURRENT_DATE, v_product_code, p_quantity,
(v_price * p_quantity));

        -- Update stock details
```

```
UPDATE products
SET quantity_remaining = quantity_remaining - p_quantity,
    quantity_sold = quantity_sold + p_quantity
WHERE product_code = v_product_code;

-- Confirmation message
RAISE NOTICE 'PRODUCT SOLD..! Order placed successfully
for % unit(s) of %.', p_quantity, p_product_name;

ELSE
    -- Step 3: If stock is insufficient
    RAISE NOTICE 'INSUFFICIENT QUANTITY..! Order cannot be
processed for % unit(s) of %.', p_quantity, p_product_name;
END IF;
END;
$$;

CALL pr_buy_products('Samsung Galaxy S23 Ultra', 3);
```